

## Lab 7

### Programing with Python Lab

#### BCACC393

#### Part A — Object-Oriented Programming Using Python

##### Questions 1–15

###### 1. Class and Objects

Create a class `Student` with attributes `name`, `roll_no`, and `marks`. Create three objects of the class and display the details of all students.

###### 2. Constructor

Create a class `Employee` with a constructor that initializes the employee's `name`, `employee_id`, `department`, and `salary`. Create an object and display all the details.

###### 3. Instance and Class Attributes

Create a class `Employee` with a class attribute `company = "ABC Technologies"` and instance attributes `name` and `salary`. Create multiple objects and demonstrate the difference between class attributes and instance attributes.

###### 4. Instance Methods

Create a class `Rectangle` with `length` and `width` as instance attributes. Define methods to calculate the **area**, **perimeter**, and **diagonal** of the rectangle.

###### 5. Private Attributes and Methods

Create a class `BankAccount` containing private attributes `__account_no` and `__balance`. Define private method `__calculate_interest()` and public methods for depositing money, withdrawing money, and displaying the balance.

###### 6. Inheritance

Create a base class `Person` with attributes `name` and `age`. Create a derived class `Student` with additional attributes `roll_no` and `course`. Use inheritance to display all student details.

###### 7. Multilevel Inheritance

Create three classes `Vehicle`, `Car`, and `ElectricCar` to demonstrate multilevel inheritance. Each class should have at least one unique method, and the `ElectricCar` object should be able to access methods from all three classes.

###### 8. Multiple Inheritance

Create two classes `Father` and `Mother`, each containing a method representing a skill. Create a class `Child` that inherits from both classes and demonstrate multiple inheritance.

## **9. Hierarchical Inheritance**

Create a base class `Shape` with a method `display()`. Create three derived classes `Circle`, `Rectangle`, and `Triangle`, each having its own method to calculate the corresponding area.

## **10. Method Overriding**

Create a base class `Animal` with a method `sound()`. Create derived classes `Dog`, `Cat`, and `Cow` and override the `sound()` method in each class with an appropriate sound.

## **11. Operator Overloading**

Create a class `Distance` with attributes `feet` and `inches`. Overload the `+` operator using `__add__()` to add two `Distance` objects and display the resulting distance.

## **12. Method Overloading Using Default Arguments**

Create a class `Calculator` with an `add()` method that can perform addition of two or three numbers using default arguments. Demonstrate both cases.

## **13. Abstract Base Class (ABC)**

Create an abstract class `Shape` with an abstract method `area()`. Create derived classes `Circle`, `Rectangle`, and `Triangle` and implement the `area()` method in each class.

## **14. Polymorphism**

Create classes `Circle`, `Rectangle`, and `Triangle`, each having an `area()` method. Write a function that accepts different shape objects and calls their `area()` method to demonstrate **runtime polymorphism**.

## **15. Metaclass**

Create a custom metaclass `MyMeta` that automatically adds a class attribute `category = "Python Class"` whenever a new class is created using the metaclass. Create two classes using this metaclass and display the automatically added attribute.

## **Part B — Errors, Exceptions and File Handling**

### **16. Handling Division by Zero**

Write a Python program that accepts two numbers from the user and performs division. Use `try-except` to handle the `ZeroDivisionError` when the denominator is zero.

### **17. Multiple Exception Handling**

Write a program that accepts two integers and performs division. Handle both `ValueError` and `ZeroDivisionError` using appropriate exception-handling blocks.

### **18. try-except-else-finally**

Write a Python program that accepts an integer from the user and calculates its square. Use `try`, `except`, `else`, and `finally` blocks to demonstrate the complete exception-handling mechanism.

### **19. Raising User-Defined Exceptions**

Create a user-defined exception `InvalidAgeError`. Write a program that accepts a person's age and raises this exception if the age is less than 18.

## 20. Handling Multiple Exceptions

Write a program that accepts two values and performs arithmetic operations. Handle possible exceptions such as invalid input, division by zero, and other appropriate runtime errors.

## 21. Create and Write to a Text File

Write a Python program to create a file named `student.txt` and write the following information into it:

- Student name
- Roll number
- Course
- Marks

Use appropriate file-opening and closing techniques.

## 22. Read Data from a File

Write a Python program to open `student.txt`, read its contents, and display them on the screen. Demonstrate both `read()` and `readline()` methods.

## 23. Count Lines, Words, and Characters

Write a Python program to read a text file and count the **number of lines, words, and characters** present in the file.

## 24. Copy File Contents

Write a Python program to read the contents of `source.txt` and copy them into another file named `destination.txt`. Display a suitable message after successful copying.

## 25. File Handling with Exception Handling

Write a Python program that asks the user for a filename and attempts to open and read the file. Handle `FileNotFoundError`, `PermissionError`, and other appropriate exceptions. Use a `finally` block to display a message indicating that the file operation has completed.